File Edit Selection View Go Run Terminal Help

chatui assignmnet

EXPLORER

CHATUI ASSIGNMNET

- chat.js
- ChatUI_Submission.zip
- cohere_example.py 1
- gemini_example.py
- groq_example.py
- huggingface_example.py
- index.html
- ollama_example.py
- openai_example.py
- README.md
- requirements.txt
- style.css

OUTLINE

TIMELINE

cohere_example.py 1

cohere_example.py > ...

```python
# cohere_example.py
# Queries Cohere models using the Cohere Python SDK

import os
import cohere

# Configure API - reads key from environment variable
api_key = os.getenv("COHERE_API_KEY")
client = cohere.Client(api_key=api_key)


def query_cohere(prompt):
    """Query Cohere model with a user prompt."""
    try:
        response = client.chat(
            model="command-r",   # reliable model available on free tier
            message=prompt,
            temperature=0.7,
            max_tokens=500
        )
        return response.text

    except Exception as e:
        return f"Error: {str(e)}"


# Main execution
if __name__ == "__main__":
    user_prompt = input("Enter your prompt: ")
    print("\nQuerying Cohere...")
    result = query_cohere(user_prompt)
    print("\nResponse:")
    print(result)
```

PROBLEMS 1 OUTPUT DEBUG CONSOLE TERMINAL PORTS

powershell

```
PS C:\chatui assignmnet>
```

CHAT

SESSIONS

cohere_example.py

Describe what to build

Auto

Launched ⊗ 0 ⚠ 1 Update is ready, click to restart.

Ln 34, Col 1 Spaces: 4 UTF-8 LF {} Python 3.11.9 (Microsoft Store) Python 3.11

30°C Sunny

Search

ENG IN 11:17 13/04/2026

File Edit Selection View Go Run Terminal Help

chatui assignmnet

EXPLORER

CHATUI ASSIGNMNET

- chat.js
- ChatUI_Submission.zip
- cohere_example.py
- gemini_example.py 1
- groq_example.py
- huggingface_example.py
- index.html
- ollama_example.py
- openai_example.py
- README.md
- requirements.txt
- style.css

OUTLINE

TIMELINE

gemini_example.py 1

gemini_example.py > ...

```python
# gemini_example.py
# Queries Google Gemini using the google-generativeai SDK

import os
import google.generativeai as genai

# Configure API - reads key from environment variable
api_key = os.getenv("GOOGLE_API_KEY")
genai.configure(api_key=api_key)

# Initialize the Gemini model
model = genai.GenerativeModel("gemini-1.5-flash")   # free tier friendly model


def query_gemini(prompt):
    """Query Google Gemini with a user prompt."""
    try:
        response = model.generate_content(
            prompt,
            generation_config=genai.types.GenerationConfig(
                max_output_tokens=500,
                temperature=0.7
            )
        )
        return response.text

    except Exception as e:
        return f"Error: {str(e)}"


# Main execution
if __name__ == "__main__":
    user_prompt = input("Enter your prompt: ")
    print("\nQuerying Google Gemini...")
    result = query_gemini(user_prompt)
    print("\nResponse:")
    print(result)
```

CHAT

SESSIONS

gemini_example.py

Describe what to build

Auto

Launched ⊗ 0 ⚠ 1 Update is ready, click to restart.

Ln 35, Col 39 Spaces: 4 UTF-8 LF {} Python 3.11.9 (Microsoft Store) Python 3.11

30°C Sunny

Search

ENG IN 11:17 13/04/2026

File Edit Selection View Go Run Terminal Help

chatui assignmnet

EXPLORER

CHATUI ASSIGNMNET

- chat.js
- ChatUI_Submission.zip
- cohere_example.py
- gemini_example.py
- groq_example.py 1
- huggingface_example.py
- index.html
- ollama_example.py
- openai_example.py
- README.md
- requirements.txt
- style.css

groq_example.py 1

groq_example.py > ...

```python
# groq_example.py
# Queries Groq-hosted Llama models using the Groq Python SDK

import os
from groq import Groq

# Configure API - reads key from environment variable
api_key = os.getenv("GROQ_API_KEY")
client = Groq(api_key=api_key)


def query_groq(prompt):
    """Query Groq Llama model with a user prompt."""
    try:
        response = client.chat.completions.create(
            model="llama3-8b-8192",  # fast and free on Groq's free tier
            messages=[
                {"role": "user", "content": prompt}
            ],
            max_tokens=500,
            temperature=0.7
        )
        return response.choices[0].message.content

    except Exception as e:
        return f"Error: {str(e)}"


# Main execution
if __name__ == "__main__":
    user_prompt = input("Enter your prompt: ")
    print("\nQuerying Groq...")
    result = query_groq(user_prompt)
    print("\nResponse:")
    print(result)
```

CHAT

SESSIONS

groq_example.py

Describe what to build

Auto

OUTLINE

TIMELINE

Launchpad ⊗ 0 ⚠ 1 Update is ready, click to restart.

Ln 1, Col 1 Spaces: 4 UTF-8 LF {} Python 3.11.9 (Microsoft Store) Python 3.11

30°C Sunny

Search

ENG IN 11:17 13/04/2026

File Edit Selection View Go Run Terminal Help

chatui assignmnet

EXPLORER

CHATUI ASSIGNMNET

- chat.js
- ChatUI_Submission.zip
- cohere_example.py
- gemini_example.py
- groq_example.py
- huggingface_example.py 1
- index.html
- ollama_example.py
- openai_example.py
- README.md
- requirements.txt
- style.css

huggingface_example.py 1

huggingface_example.py > ...

```python
# huggingface_example.py
# Queries Hugging Face Inference API using the requests library

import os
import requests

# Configure API - reads key from environment variable
API_KEY = os.getenv("HUGGINGFACE_API_KEY")

# Using a reliable, free text-generation model
MODEL_ID = "mistralai/Mistral-7B-Instruct-v0.1"
API_URL = f"https://api-inference.huggingface.co/models/{MODEL_ID}"

HEADERS = {"Authorization": f"Bearer {API_KEY}"}


def query_huggingface(prompt):
    """Query Hugging Face Inference API with a user prompt."""
    try:
        payload = {
            "inputs": prompt,
            "parameters": {
                "max_new_tokens": 500,
                "temperature": 0.7,
                "return_full_text": False   # return only the generated part
            }
        }

        response = requests.post(API_URL, headers=HEADERS, json=payload, timeout=60)
        response.raise_for_status()

        data = response.json()

        # The response is a list of generated text objects
        if isinstance(data, list) and len(data) > 0:
            return data[0].get("generated_text", "No text generated.")
        else:
            return str(data)

    except Exception as e:
        return f"Error: {str(e)}"


# Main execution
if __name__ == "__main__":
    user_prompt = input("Enter your prompt: ")
    print("\nQuerying Hugging Face...")
    result = query_huggingface(user_prompt)
```

CHAT

SESSIONS

huggingface_example.py

Describe what to build

Auto

OUTLINE

TIMELINE

Launchpad 0 1 Update is ready, click to restart.

Ln 1, Col 1 Spaces: 4 UTF-8 LF {} Python 3.11.9 (Microsoft Store) Python 3.11

30°C Sunny

Search

ENG IN 11:17 13/04/2026

File Edit Selection View Go Run Terminal Help

chatui assignmnet

EXPLORER

CHATUI ASSIGNMNET

- chat.js
- ChatUI_Submission.zip
- cohere_example.py
- gemini_example.py
- groq_example.py
- huggingface_example.py
- index.html
- ollama_example.py 1
- openai_example.py
- README.md
- requirements.txt
- style.css

OUTLINE

TIMELINE

ollama_example.py 1

ollama_example.py > ...

```python
# ollama_example.py
# Queries a locally running Ollama model via its REST API
# Ollama must be installed and running: https://ollama.ai/
# Pull a model first with: ollama pull llama3

import requests
import json

# Ollama runs locally — no API key needed
OLLAMA_URL = "http://localhost:11434/api/generate"
MODEL_NAME = "llama3"   # change to any model you have pulled locally


def query_ollama(prompt):
    """Query a local Ollama model with a user prompt."""
    try:
        payload = {
            "model": MODEL_NAME,
            "prompt": prompt,
            "stream": False   # get a single complete response
        }

        response = requests.post(OLLAMA_URL, json=payload, timeout=120)
        response.raise_for_status()

        # Parse the JSON response
        data = response.json()
        return data.get("response", "No response received.")

    except requests.exceptions.ConnectionError:
        return "Error: Could not connect to Ollama. Make sure Ollama is running (run 'ollama serve')."
    except Exception as e:
        return f"Error: {str(e)}"


# Main execution
if __name__ == "__main__":
    user_prompt = input("Enter your prompt: ")
    print(f"\nQuerying Ollama ({MODEL_NAME})...")
    result = query_ollama(user_prompt)
    print("\nResponse:")
    print(result)
```

CHAT

SESSIONS

ollama_example.py

Describe what to build

Auto

Default Approv...

Launched ⊗ 0 ⚠ 1 Update is ready, click to restart.

Ln 1, Col 1 Spaces: 4 UTF-8 LF {} Python 3.11.9 (Microsoft Store) Python 3.11

30°C Sunny

Search

ENG IN 11:17 13/04/2026

File Edit Selection View Go Run Terminal Help

chatui assignmnet

EXPLORER

CHATUI ASSIGNMNET

- chat.js
- ChatUI_Submission.zip
- cohere_example.py
- gemini_example.py
- groq_example.py
- huggingface_example.py
- index.html
- ollama_example.py
- openai_example.py 1
- README.md
- requirements.txt
- style.css

OUTLINE

TIMELINE

openai_example.py 1

openai_example.py > ...

```python
# openai_example.py
# Queries OpenAI GPT models using the OpenAI Python SDK

import os
from openai import OpenAI

# Configure API - reads key from environment variable
api_key = os.getenv("OPENAI_API_KEY")
client = OpenAI(api_key=api_key)


def query_openai(prompt):
    """Query OpenAI GPT model with a user prompt."""
    try:
        response = client.chat.completions.create(
            model="gpt-3.5-turbo",   # change to "gpt-4o" if your plan supports it
            messages=[
                {"role": "user", "content": prompt}
            ],
            max_tokens=500,
            temperature=0.7
        )
        return response.choices[0].message.content

    except Exception as e:
        return f"Error: {str(e)}"


# Main execution
if __name__ == "__main__":
    user_prompt = input("Enter your prompt: ")
    print("\nQuerying OpenAI...")
    result = query_openai(user_prompt)
    print("\nResponse:")
    print(result)
```

CHAT

SESSIONS

openai_example.py

Describe what to build

Auto

Launched ⊗ 0 ⚠ 1 Update is ready, click to restart.

Ln 1, Col 1 Spaces: 4 UTF-8 LF {} Python 3.11.9 (Microsoft Store) Python 3.11

30°C Sunny

Search

ENG IN 11:18 13/04/2026

File C:/New%20folder%20(2)/UI%20assignment%204/index.html

Chat Assistant
Professional AI conversation

+ New Chat

RECENT CHATS

Chat 2

Chat 1

Chatbot Workspace

A clean and responsive chat UI with quick prompts and modern styling.

Welcome back

Ready

Text Summary
Ask for a concise summary.

Explain Idea
Request a simple explanation.

Email Draft
Create a professional reply.

Productivity Tips
Get practical daily advice.

AI

Switched to chat: Chat 2

Send